E3 - Implementing advanced graphics with PSOC™ Edge training manual

About this document

Scope and purpose

This document is the training manual for the graphics subsystem of PSOC™ Edge E84 MCU. This training aims to achieve the following:

- Provide a detailed understanding of Hardware and associated Software features of graphics subsystem of PSOC™ Edge E84 MCU
- Optimizing the application for power and performance

Table of contents

About this document **1**
Table of contents **2**
1 Introduction **3**
2 Pre-requisites **3**
3 Required development tools **4**
4 Graphics subsystem overview **5**
4.1 Objective 5
4.2 Project creation 5
4.3 Description 5
4.4 Display controller configuration 6
4.4.1 Overlay layers 6
4.4.2 RLAD 6
4.5 MIPI DSI 6
4.5.1 Video mode 6
4.5.2 Command mode 7
4.6 Labs 7
4.6.1 LAB 1: Debugging graphics application 7
4.6.1.1 Additional modifications 10
4.6.2 LAB 2: Use RLAD HW decompression 11
4.6.2.1 Results 13
4.6.2.2 Additional modifications 14
4.6.3 LAB 3: EEZ Studio and LVGL I/O 14
4.6.3.1 Additional modifications 23
5 Graphics performance **24**
5.1 Frames per second (FPS) 24
5.1.1 Clock setup based on required FPS for video mode displays 24
5.1.2 FPS for command mode display 24
5.1.3 FPS improvement 24
5.2 LAB 3: Observe FPS/CPU usage with and without GPU 25
5.2.1 Results 26
6 Takeaways **27**
7 Conclusion **28**
8 Appendix A: KIT_PSE84_EVAL details **29**
Revision history **30**
Disclaimer **31**

1 Introduction

This manual provides a reference for the graphics advanced training with PSOC™ Edge. This also contains the step-by-step instructions for the Lab exercises for this training.

2 Pre-requisites

Install the software and obtain the hardware listed in the Required development tools section. This class will not cover basic concepts of ModusToolbox™ and PSOC™ Edge:

- For an introduction to PSOC™, including getting started with ModusToolbox™, go to:
  - https://www.infineon.com/product-information/psocdeveloper
- For PSOC™ Edge trainings, from beginner tutorials to advanced trainings, go to:
  - PSOC™ Edge E84 Training Collection

3 Required development tools

- ModusToolbox™ software v3.7 or later
  - Recommended installation via ModusToolbox™ Setup tool
- Eclipse IDE for ModusToolbox™ v2025.8.0 or later
  - Recommended installation via ModusToolbox™ Setup tool
- Edge Protect Security Suite v1.6 or later
  - Installed by ModusToolbox™ Setup tool as a dependency to ModusToolbox™ v3.7
- ModusToolbox™ Programming Tools v1.7.0 or later
  - Installed by ModusToolbox™ Setup tool as a dependency to ModusToolbox™ v3.7
- Board support package (BSP)
  - KIT_PSE84_EVAL_EPC2: v1.0.0 or later (available with ModusToolbox™ v3.7)
- Serial terminal emulator
  - Eclipse IDE for ModusToolbox™ includes a serial terminal that includes Tera Term, PuTTY, and so on.
- PSOC™ Edge E84 Evaluation Kit (KIT_PSE84_EVAL)
  - Ensure device boots from external memory:
    - BOOT SW/SW6 in 'ON/HIGH' position
  - Disable alternate serial interface configuration:
    - Tristate/Not-Connected (NC) (J20/J21) position
    - For more details on the PSOC™ Edge EVK, see Appendix A: KIT_PSE84_EVAL details
- Waveshare 4.3-inch Raspberry Pi DSI 800x480 pixel display (included with PSOC™ Edge EVK)
- YUV player application
  - Available online, or in Lab source accompanying this training.
- EEZ Studio 0.24.0

Note: Other tool versions and hardware can be used with minimal or no modifications to the training manual.

4 Graphics subsystem overview

4.1 Objective

The objective of this section to provide a comprehensive overview of the hardware interfacing of the Graphics subsystem available within the PSOC™ Edge E84 MCU.

4.2 Project creation

The project creation of graphics applications is covered in the introductory training. Users are encouraged to refer to the introductory training for familiarity with graphics applications in ModusToolbox™ for PSOC™ Edge.

4.3 Description

The Graphics subsystem of the PSOC™ Edge E84 MCU has the following three blocks:

- 2.5D graphics processing unit (GPU)
- Display controller (DC)
- MIPI DSI host controller

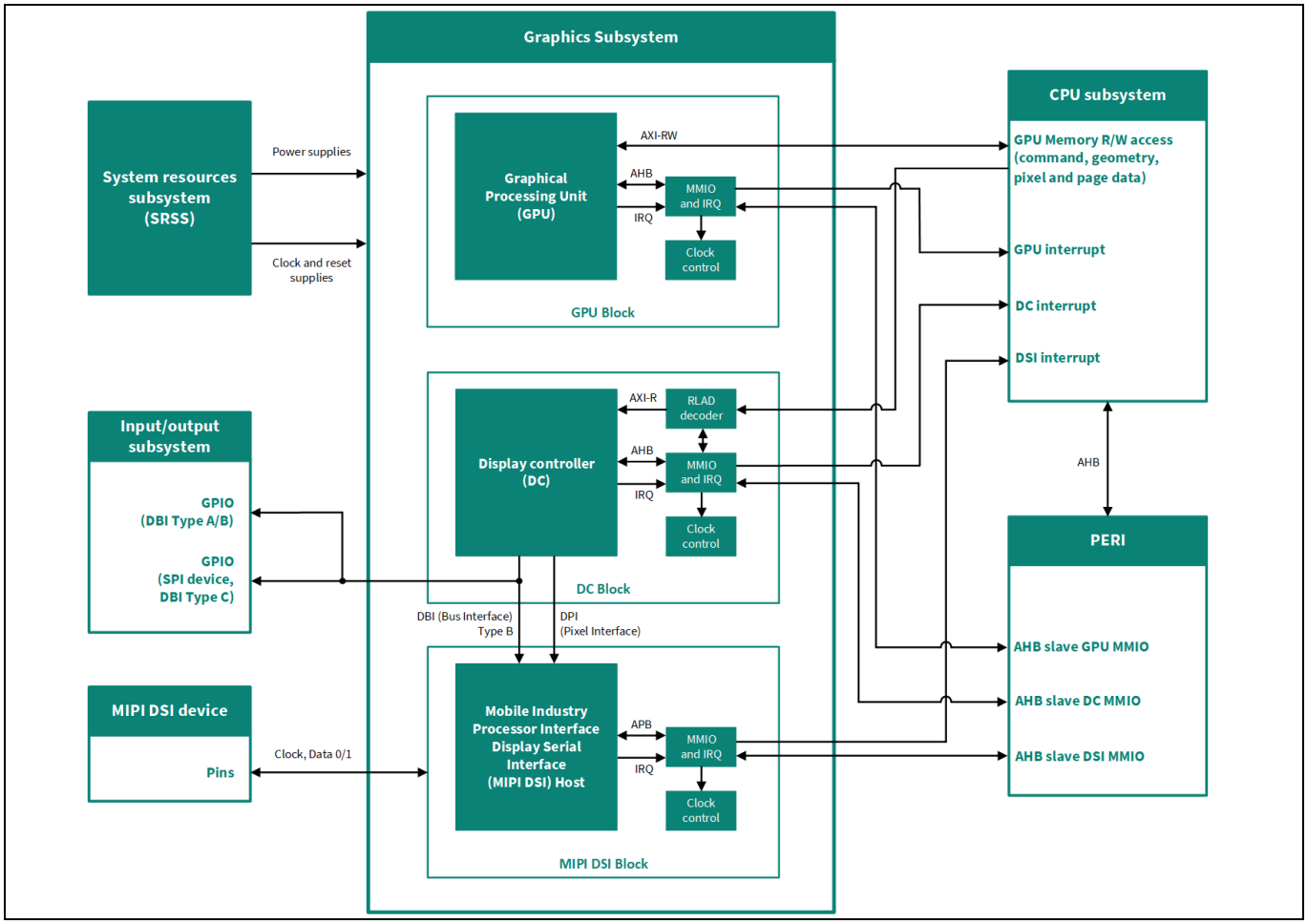


Figure 1 Graphics subsystem overview

4.4 Display controller configuration

The Display controller reads pixel data from the interfaced memory which can be internal SRAM, System SRAM (SoCMEM) or external memory such as flash or PSRAM. The output of the display controller can then be rendered over multiple formats like GPIO (DBI Type A/B), GPIO (SPI device DBI Type C) and MIPI DSI interface.

The **Display type** parameter is used to configure the type of display interfaced to the MCU. The below figure lists the various display types supported by the PSOC™ Edge E84 MCU.

The output of the display controller can be rendered over DBI Type A, Type B and Type C protocols on dedicated IOs. The DC output is routed to the DSI host controller in DBI or DPI format. The **Display Interface** parameter is then set based on the display panel requirements.

Users are encouraged to observe the various combinations of **Display Types** and **Display Interfaces** available in the PSOC™ Edge E84 MCU.

4.4.1 Overlay layers

The Display controller supports rendering on three layers. The use of overlay layers is explained in detail in the E2 introductory training, which includes hands-on lab sessions.

4.4.2 RLAD

The display controller of PSOC™ Edge E84 MCU included a hardware RLAD decoder which can be used to decompress and render images on-the-fly. This feature is mainly used for compressing static image data which can be decompressed, blended with dynamic content and then rendered to the display thereby reducing static memory consumption.

The LAB exercise in section 4.6.2 covers the usage of RLAD.

4.5 MIPI DSI

MIPI DSI (Mobile Industry Processor Interface, Display Serial Interface) is a high-speed, cost-effective protocol designed by the MIPI Alliance for connecting display panels to host processor systems in mobile and embedded devices.

The MIPI DSI Host sits between the MCU's display controller and the panel, converting rendered pixels and control commands into DSI packets that the D-PHY transmits over one or more serial lanes. The DSI host transmits the DSI packets in the form of parallel data to the D-PHY through the PHY protocol interface (PPI). The D-PHY serializes the packets and sends them across the serial link.

4.5.1 Video mode

In Video Mode, DSI host receives a continuous pixel stream via **DPI** (DPI-24/18/16), packetizes per the selected transfer type (non-burst pulses/events or burst) from the DC, and transmits in High-Speed with the required clock-lane behaviour.

To render a stable image, it is important to accurately set the video timings compatible with the display being driven by the MCU. MIPI DSI has the following video timing signals.

Table 1 Timing parameters

Timing parameter	Description
Hsync width	Duration of the horizontal sync pulse, signalling the start of a new line
HFP	Idle period after the active pixels before the horizontal sync pulse. Used by the display to prepare for the next line.
HBP	Idle period after the horizontal sync pulse before starting to send active data pixels.
Vsync width	Number of lines during the active vertical sync pulse.
VFP	Number of idle lines after completing active frame lines before the vertical sync pulse.
VBP	Number of idle lines after the VSync pulse before starting the next frame.

4.5.2 Command mode

In Command Mode, it ingests panel commands and windowed pixel bursts via **DBI**, using Low-Power packets for control and High-Speed long writes for image updates. This mode is particularly useful for low power application which requires lower refresh rates such as smartwatch and wearable devices.

For more detailed information about various configurable parameters of the graphics subsystem, see the application notes Getting started with graphics on PSOC™ Edge MCU and Developing high-performance graphics with low-power optimization on POSC Edge MCU.

4.6 Labs

4.6.1 LAB 1: Debugging graphics application

The process of debugging any application is inherently complex and often an open-ended one. While it is impossible to cover every scenario, this section attempts to cover a common debugging technique to help guide the user troubleshooting efforts.

Note: Use of Eclipse IDE is required for this lab as some features like exporting memory contents are not available in VSCode based extensions.

1. Start Eclipse IDE 2025.8 or newer
2. Create new workspace “Graphics Training” or select existing training workspace. Create *PSOC_Edge_Graphics_using_VGLite_API* code example
3. Open *proj_cm55/main.c* and place a breakpoint when calling `Cy_GFXSS_Set_FrameBuffer` inside `swap_frame_buffer( )`

```
817 /*******************************************************************************
818 * Function Name: swap_frame_buffer
819 ********************************************************************************
820 * Summary:
821 *  This function waits for any pending frame buffer operation, sets the
822 *  Video/Graphics layer frame buffer to the display controller, and swaps
823 *  between two render buffers. It also updates the FPS statistics.
824 *
825 * Parameters:
826 *  void
827 *
828 * Return:
829 *  void
830 *
831 *******************************************************************************/
832 void swap_frame_buffer( void )
833 {
834     static int current_buffer = RESET_VAL;
835
836     while (fb_pending);
837
838     fb_pending = true;
839
840     /* Sets Video/Graphics layer buffer address and transfers the frame buffer
841        to the Display Controller */
842     Cy_GFXSS_Set_FrameBuffer(base, (uint32_t*) render_target->address, &gfx_context);
843
844     /* Swap buffers */
845     render_target = (current_buffer) ? &buffer0 : &buffer1;
846     current_buffer ^= 1;
847
848     /* Update frame rate statistics */
849     calculate_fps();
850 }// swap_frame_buffer
851
```

Figure 2 Adding breakpoint

4. Add the “render_target” variable to the “Expressions” tab

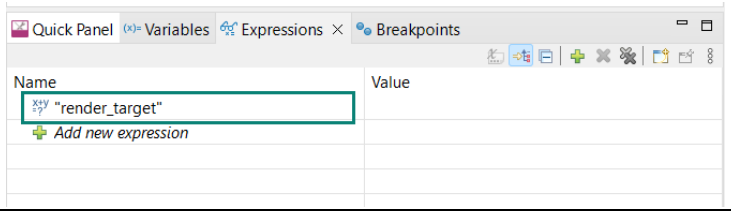


Figure 3 Adding expression to watch window

5. Now build and program the application
6. Launch the CM55 debug config. Once the debug session starts, keep pressing the resume button till a frame is rendered on the display
7. Now note the value of the “memory” variable which is part of the “render_target” structure. Add this hex value to the memory monitor taps

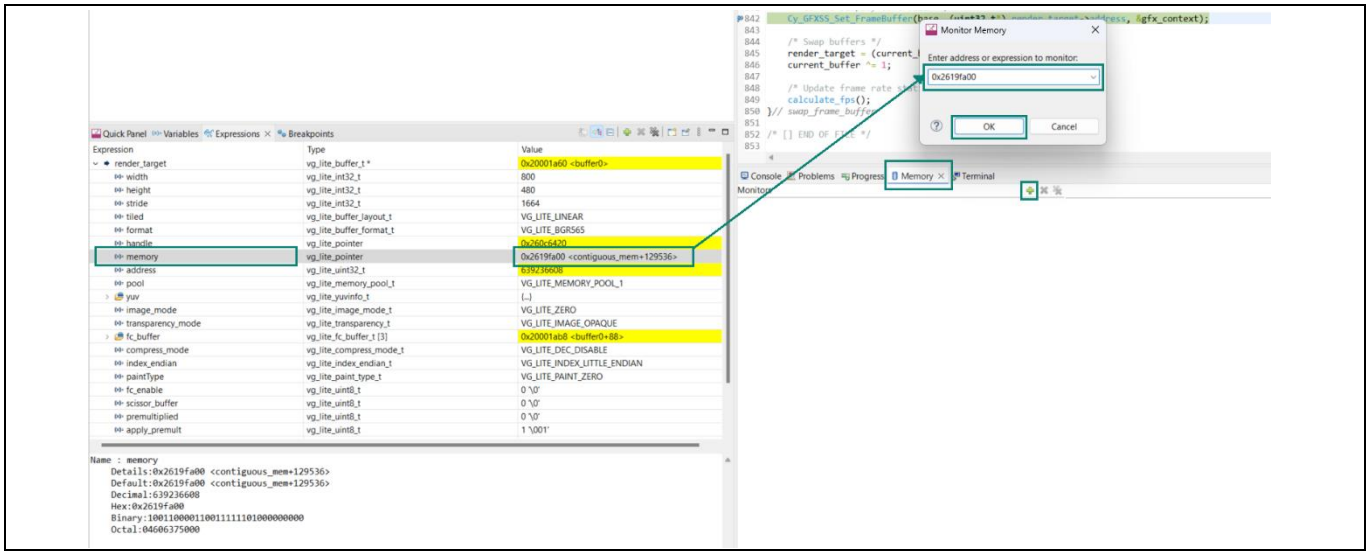


Figure 4 Copy the frame buffer address to memory monitor

8. Export the memory contents using the “Export” option. Set the following options in the export memory tab. Then Click “Ok”
   a. Format: Raw binary
   b. Length: Display width aligned to 128B * Display height * Bytes per pixel (832 * 480 * 2)
   c. File name: Name of binary file with .bin extension to which data will be exported

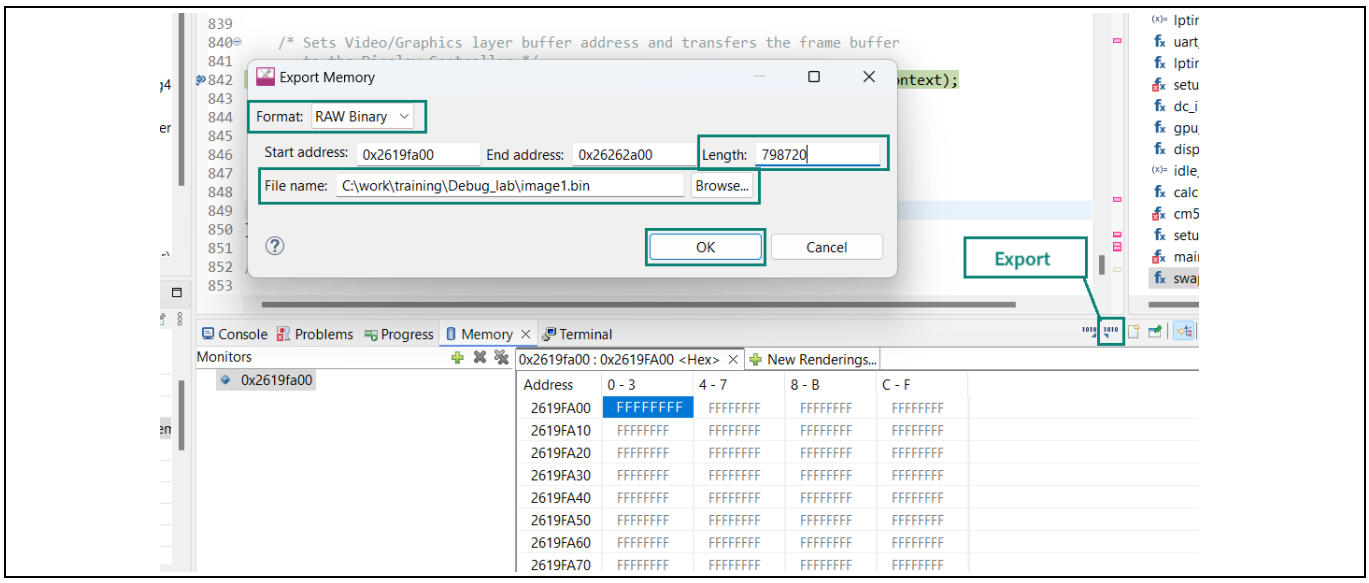


Figure 5 Export frame buffer data to binary file

9. Observe that a binary file is created at the location given in the above step with the given file name
10. Open the YUV player application and open the binary file created after the Step-8
    Note: the YUV player is available in the lab source accompanying this training at: <lab source>/LAB1_Graphics_Debug/yuvplayer.exe
11. Select Size → Custom. Set the Width to 832 and Height to 480

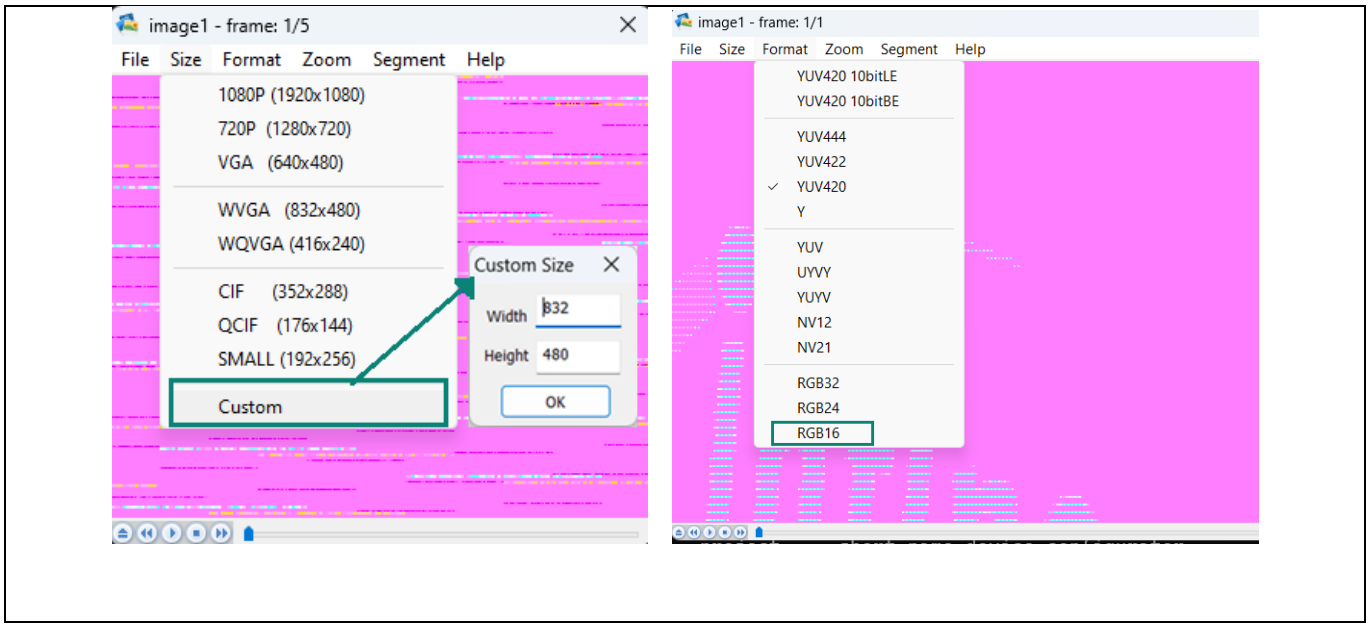


Figure 6 YUV player settings

12. Next, set the Format to RGB16. The YUV player displays the image from the pixel data provided in the binary file

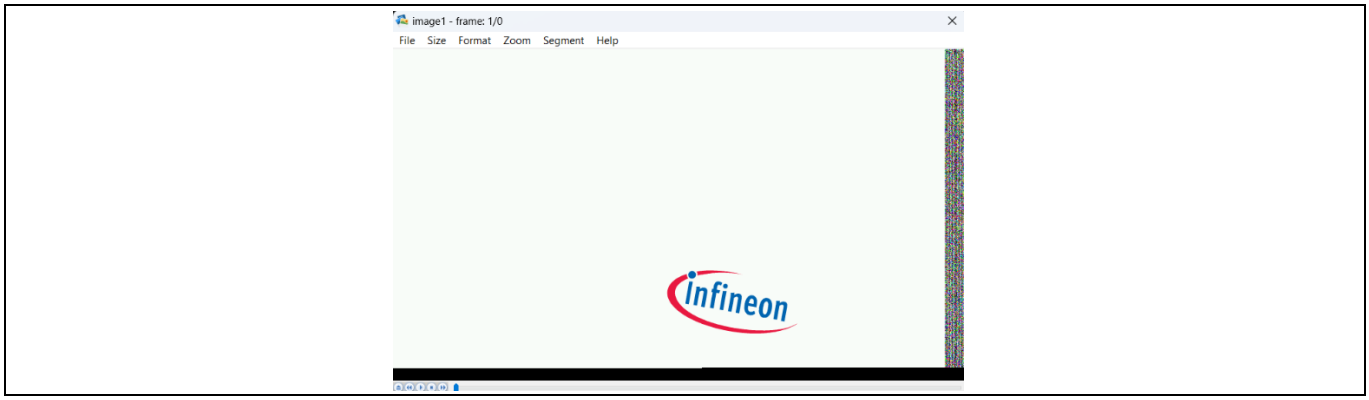


Figure 7 Frame buffer data visualized in YUV player

Note:

1. *This is the next frame which is composed by the GPU and to be rendered by the DC*
2. *The frame composed by GPU has an image of resolution 800 * 480. Since we are using RGB565 (16-bit) format, the vglite buffer's width is aligned to 128 bytes. This is evident from the garbage data towards the end of the frame displayed above*

4.6.1.1 Additional modifications

1. Furthermore, the user can view the register list of the Graphics subsystem by enabling the "GFXSS" in "Peripherals" tab

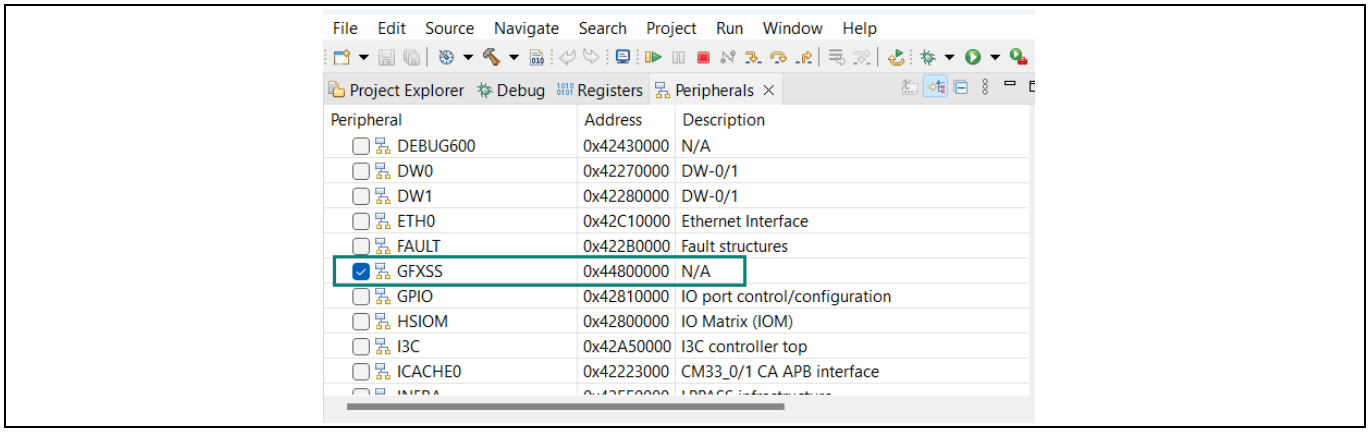


Figure 8 Enable GFXSS register view

2. Select the “Memory” tab to view the complete register list of graphics subsystem of PSOC™ Edge E84

Note: This feature is useful to find out the exact cause of error using status registers of GPU, DC and DSI controller.

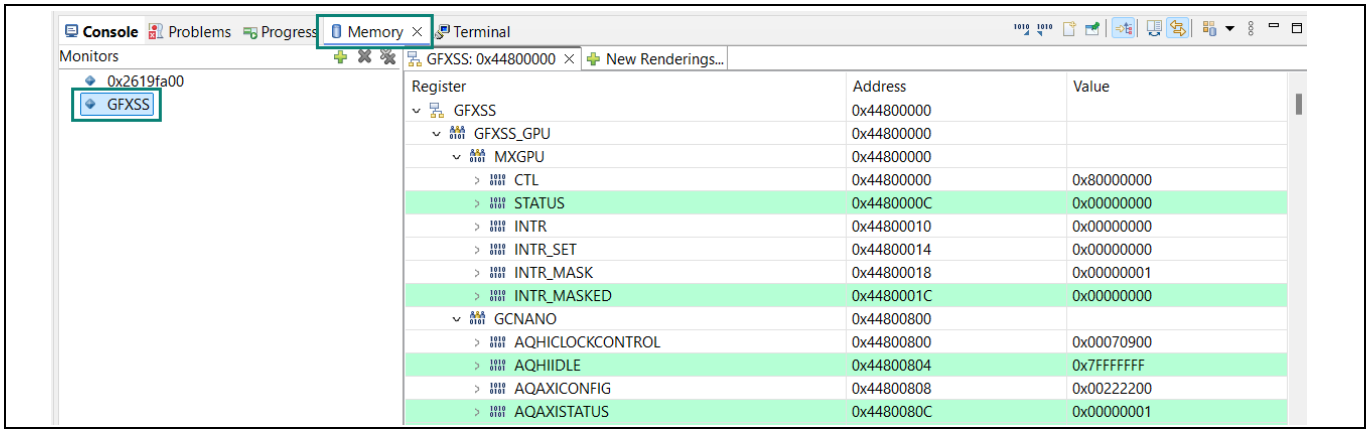


Figure 9 Register list of graphics subsystem of PSOC™ Edge E84

4.6.2 LAB 2: Use RLAD HW decompression

In this exercise, we will use the RLAD hardware decompression feature of the display controller of the PSOC™ Edge E84 MCU. The source image (in png format) is compressed to a header file using the *rlad-encoder* script which is provided in the utility folder of *PSOC_Edge_Graphics_using_RLAD* the code example (Figure 11). The image is then decompressed and rendered on-the-fly by the RLAD decoder.

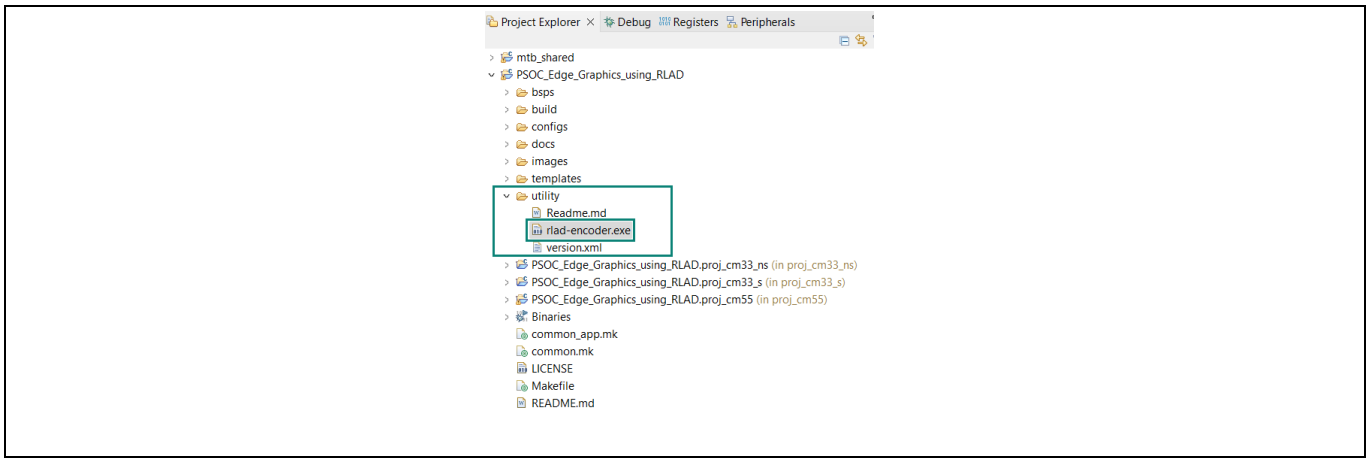


Figure 10 rlad-encoder script location

The various compression formats supported on PSOC™ Edge E84 are as follows:

Table 2 Compression formats

–	**RLAD** **run-length adaptive dithering**	**RLAD_UNIFORM** **Uniform package size**	**RLA** **run-length adaptive**	**RL** **standard run-length**
Quality	Lossy	Lossy	Lossless	Lossless
Compression ratio	High	High	Low	High

1. Create *PSOC_Edge_Graphics_using_RLAD* code example in existing workspace
2. In this exercise we will compress the *ifx_logo.png* image with RL, RLA and RLAD compression. Use the following snippet in **modus-shell** to do the same:

Code listing 1 Compression commands in modus-shell

```
/* For RL compression */
./rlad-encoder ifx_logo.png ifx_logo_RL.h RL

/* For RLA */
./rlad-encoder ifx_logo.png ifx_logo_RLA.h RLA

/* For RLAD */
./rlad-encoder ifx_logo.png ifx_logo_RLAD.h RLAD
```

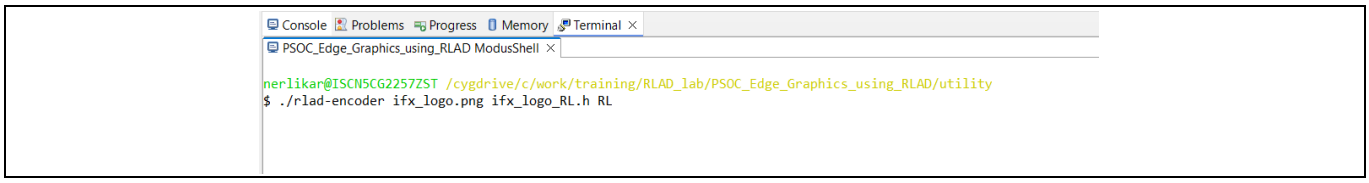


Figure 11 rlad-encoder script using modus-shell

3. After the above step, three header files should be created with- *ifx_logo_RL.h*, *ifx_logo_RLA.h* and *ifx_logo_RLAD.h*. Place these header files in the *proj_cm55/* folder

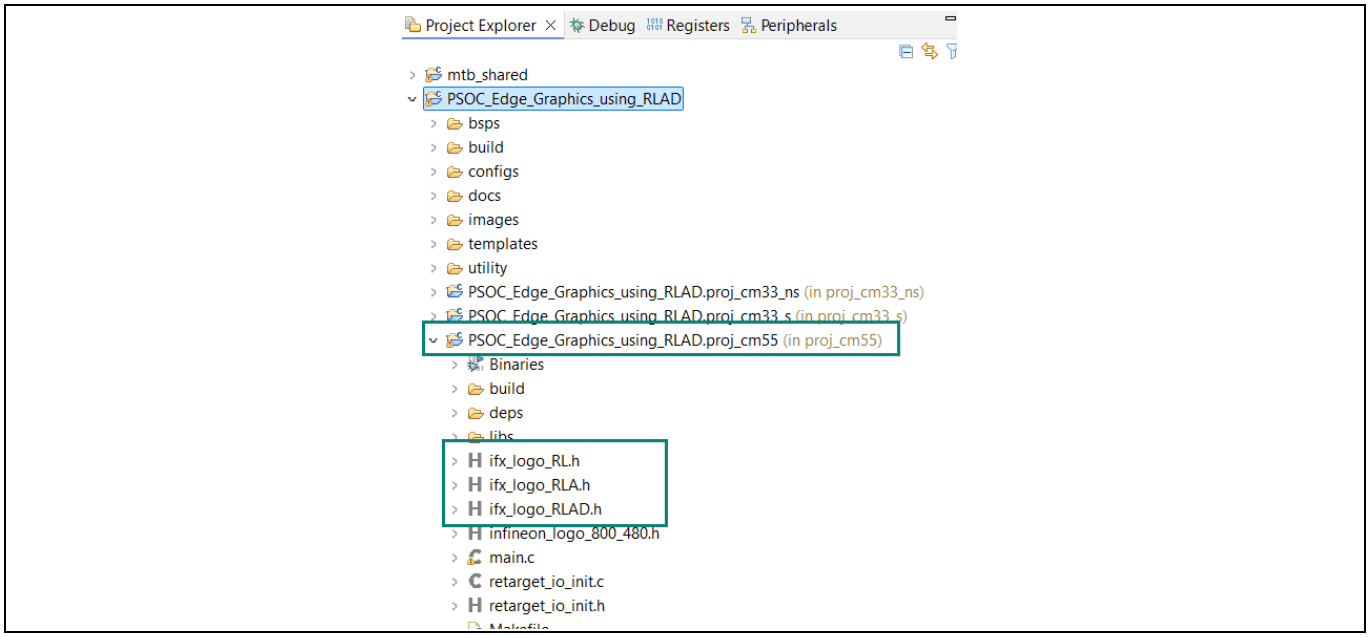


Figure 12 Compressed images generated in proj_cm55 folder

4. Replace the *main.c* file with the one provided in the lab source folder
5. In the *main.c* file set the CONFIG_COMPRESSION_MODE macro to 1 to observe the RL compression. Build and program the application
6. Observe the rendered image. Also note the original image size, compressed size and compression rate logged on terminal

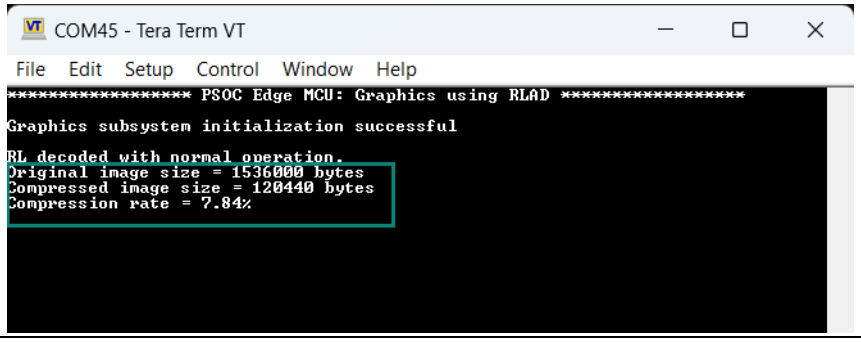


Figure 13 Terminal output for RL compression

7. Follow the Step-6 and Step-7 with values 2 and 3 for CONFIG_COMPRESSION_MODE for RLA and RLAD compressions

4.6.2.1 Results

Table 3 Compression results

Compression format	RL (lossy)	RLA (lossy)	RLAD (lossless)
Original image size (in bytes)	1536000	1536000	1536000
Compressed image size (in bytes)	120440	163552	418484
Compression ratio (%) (Compressed size/original size)	7.84	10.65	27.25

- The reduction in image size is more in lossy compression (RL, RLA) as compared to lossless compression (RLAD)
- These results will vary based on the input image supplied to the script

4.6.2.2 Additional modifications

Try the above steps with the following images: city.png, city1.png, city2.png, farm.png.
The following are the obtained results:

Table 4 Results of compressing other files

Image	RLAD (Lossy)		RL (Lossless)		RLA (Lossless)	
	File size	Rate	File size	Rate	File size	Rate
Logo	418,484	27.25%	120,440	7.8%	163,552	10.65%
City 1	1120936	73%	1,522,852	99%	944,884	61%
City 2	1197104	77%	1,499,280	97%	1,027,784	66%
Farm	1002604	65%	1,477,704	96%	812,004	52%

4.6.3 LAB 3: EEZ Studio and LVGL I/O

In this exercise we will use EEZ Studio tool to add a button and a status bar to the screen. The button will control a physical LED on the EVK, and the Bar position will be controlled by the position of the Potentiometer on the EVK. This shows both input and output from the GUI.

Hints for the advanced modification

- Use EEZ Studio to generate a new UI
- Add the generated UI files to the cm55 project
- Call the corresponding functions to initialize the custom UI and update it periodically

Solution

1. Create PSOC™ Edge MCU: Graphics LVGL demo example in a new or existing workspace. For an introduction to the LVGL example with PSOC™ Edge MCU please see Technical introduction to PSOC™ Edge E84 Features Chapter 4: Graphics
2. Create a folder called “ui” under the CM55 project. This will be the build directory of the EEZ Studio LVGL GUI

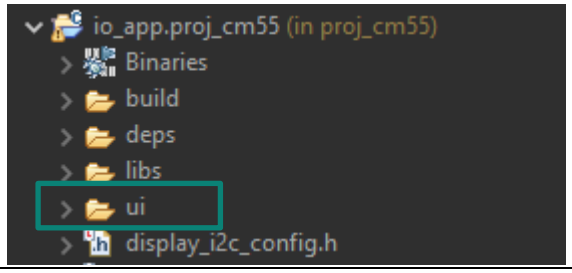


Figure 14 Adding UI folder

3. If the EEZ Studio haven't been installed, install it now from the EEZ Studio website
4. Open EEZ Studio and click on Create, select the LVGL project, select a name, add a location, and click **Create Project**

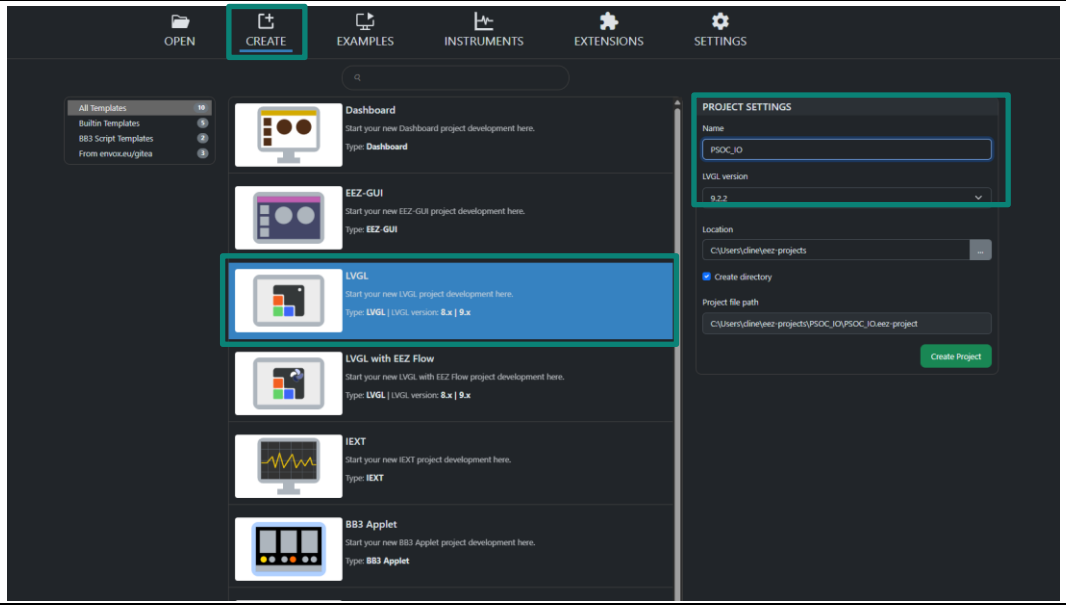


Figure 15 Creating project in EEZ Studio

5. Click the **Settings** button

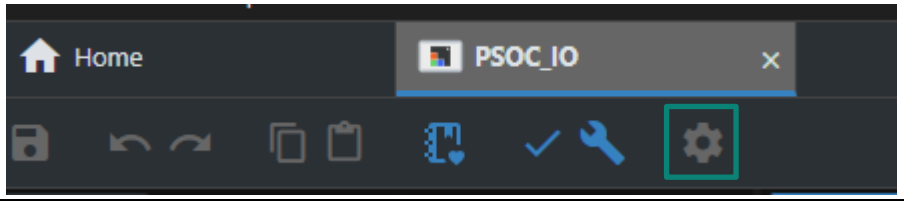


Figure 16 Open settings in EEZ Studio

6. In the **General** tab, confirm that the LVGL version is 9.2.2 since ModusToolbox™ 3.6 uses LVGL 9.2.0. User just needs to match the closest major version number. The version number that the project is using in ModusToolbox™ can be checked by going to the library manager and selecting the LVGL library. Also make sure that the display resolution is 800x480, since we are using the 4.3” display included with the PSOC™ Edge E84 kit

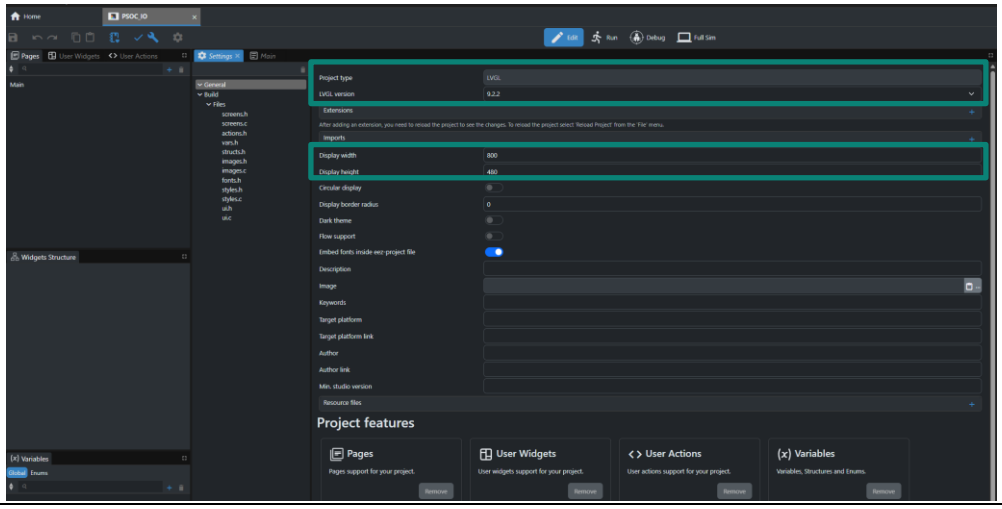


Figure 17 General settings in EEZ Studio

7. Now go to the **Build** options and modify the LVGL include setting as “lvgl.h”, since our project is configuring a direct path to LVGL inside `mtb_shared`. Change the destination folder to the “ui” folder that we created inside the CM55 project in step #2, this will put the output build directly into the project directory that will be used in our ModusToolbox™ project

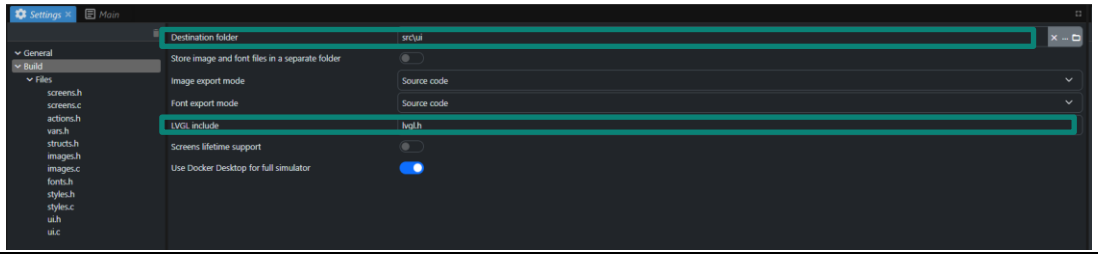


Figure 18 Build settings

8. To get back to the editing window select **Main,** and the user can now see the default page with the “Hello world” text. User may select this text and delete it by using **Delete** on the keyboard

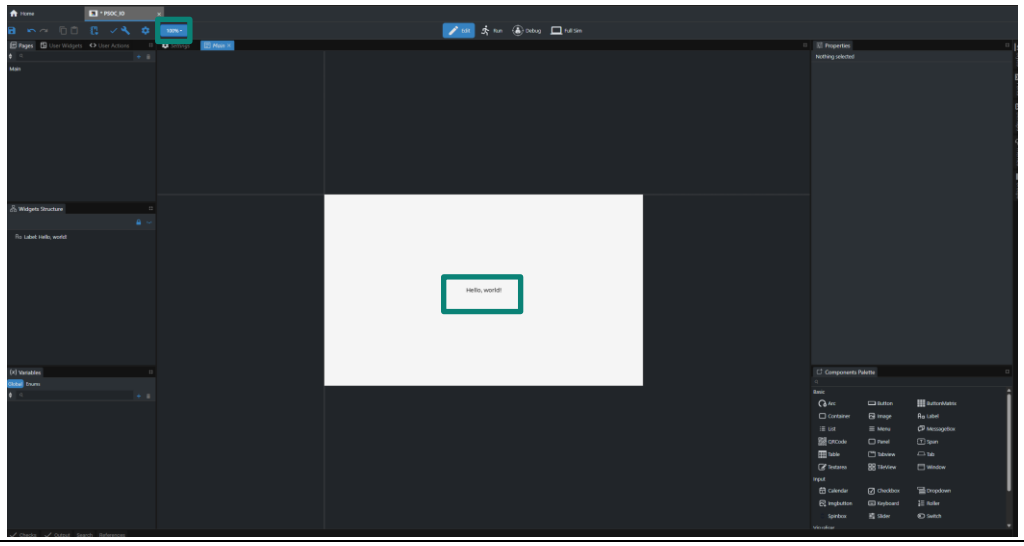


Figure 19 Back to main screen view

9. From the **Components Palette** drag in the following objects and resize for the screen: **LED**, **Button**, **Bar** and **Textarea**. We will use the Bar and the Button for this exercise, the Textarea will be used for the stretch goal of this lab. The LED can be used for users own development

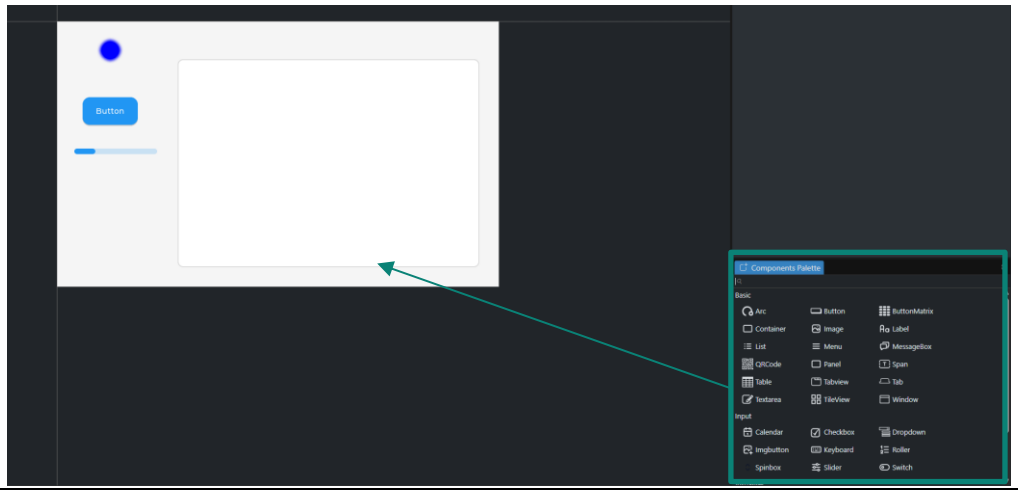


Figure 20 Selecting components

10. For each item, rename it so that it is identifiable. Note: EEZ studio will automatically format users name to follow a lowercase and underscores naming convention if necessary. If this renaming is applicable this new name will be show as “Code identifier” and will be used in place of the name typed. If the conventions are followed the “Name” will be used as normal in the code

This name is important as it identifies the component and allows user to interact with that component in users own project.

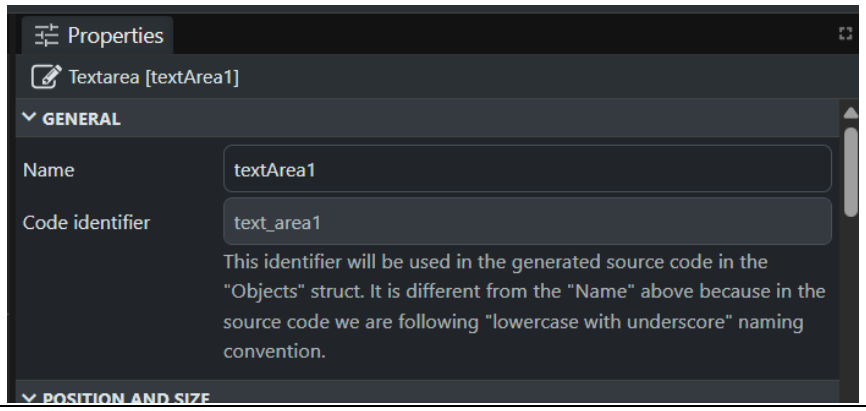


Figure 21 Adding name to components

11. Select the button that user has placed and scroll down on the right side in Properties to the **Events -> Event Handlers** section. Click the **+** to add a new event handler for the button

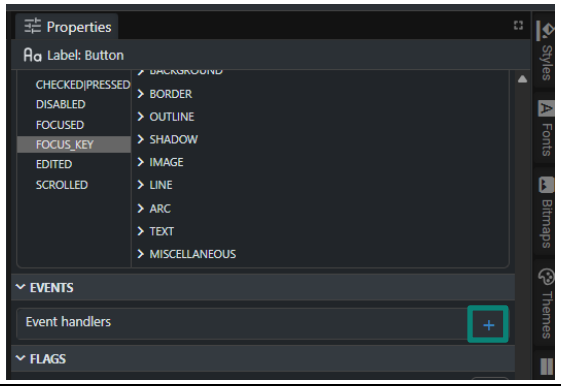


Figure 22 Adding event handler

12. The new event handler dialog will appear. Select Event: **PRESSED**. Click new action and give the action a name, we will call it **button1_pressed.** We do not need any user data so leave that as ‘0’, now click OK to create the event handler. The user will be able to see your created event in the Event handlers section on the right

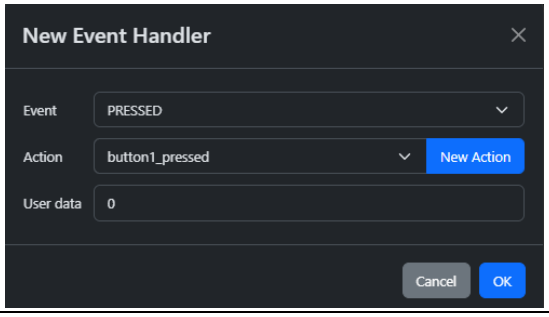


Figure 23 New event handler

13. Click the build button to build the LVGL code from EEZ Studio. This will go into the ui folder that we setup in our CM55 project directory

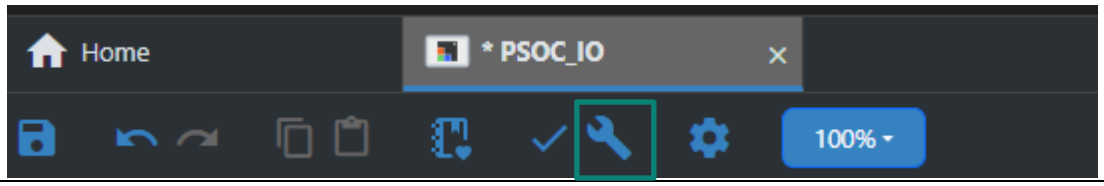


Figure 24 Build EEZ Studio project

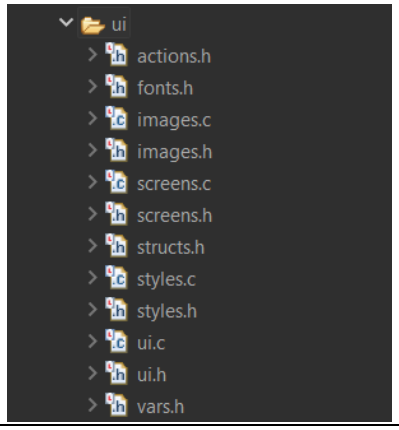


Figure 25 Build artifacts

14. Open proj_cm55/main.c and make the following modifications to initialize the new GUI and to call a periodic tick function to handle UI events

Code listing 2 Modifications to main.c to initialize custom UI

```
#include " display_i2c_config.h"
#include "ui.h"
if (VG_LITE_SUCCESS == vglite_status)
{
    /* Initialize LVGL library */
    lv_init();
    lv_port_disp_init();
    lv_port_indev_init();
    //lv_demo_music(); <-- This runs the default LVGL demo

    /* Run the custom UI */
    ui_init();
}
```

Code listing 3 Modifications to main.c to call periodic UI tick

```
for (;;)
{
    /* LVGL's timer handler function, to be called periodically to handle
* LVGL tasks.
    */
    time_till_next = lv_timer_handler();
    vTaskDelay(pdMS_TO_TICKS(time_till_next));
    ui_tick(); /* Call UI tick to handle events */
}
```

15. Next open the file actions.h in the ui folder. This has the action_button1_pressed function prototype that we setup in EEZ Studio. We must create the function for this definition. Create an actions.c file is the same ui folder. Add the following function to your file

Code listing 4 New file actions.c

```
#include "actions.h"
#include "cycfg_pins.h"
#include <lvgl.h>

void action_button1_pressed(lv_event_t * e)
{
    (void)e;
    Cy_GPIO_Inv(CYBSP_USER_LED1_PORT,CYBSP_USER_LED1_NUM);
}
```

16. Build the project and observe the USER LED1 (RED) turn on and off when the button is pressed on the screen
17. Now that we have the GUI controlling something on the EVK let have something on the EVK control the GUI. For this we will have the potentiometer (the blue knob on the EVK) control the bar on the screen

18. For this we will need a file to define the function that will set the bar percentage based on the count that we get from the ADC. Create *output_action.c* and *ouput_actions.h*. Do not make changes in *actions.h* as this is built by EEZ Studio and any user change will be overwritten

Code listing 5 New file output_actions.c

```
#include "output_actions.h"
#include "screens.h"
#include <lvgl.h>
#include <stdarg.h>
#include <stdio.h>
#include "mtb_disp_dsi_waveshare_4p3.h"
#include "display_i2c_config.h"

* Convert a raw ADC reading into a bar percentage (0..100) and update the bar.
 *
 * - adc_raw:  raw ADC counts (e.g. 0..4095 for 12-bit)
 * - adc_max:  maximum ADC count (e.g. 4095 for 12-bit, 65535 for 16-bit)
 *
 * NOTE: Change `objects.screen_bar_1` to your actual bar object name from
screens.h.
 * NOTE: Call this from the same context/task that runs lv_timer_handler().
 */
int ui_set_bar_from_adc(int adc_raw, int adc_max)
{
    if(!objects.bar1) return 100;  // bar not created yet
    if(adc_max <= 0) return 100;   //ADC Max can not be negative

    if(adc_raw < 0) adc_raw = 0;
    if(adc_raw > adc_max) adc_raw = adc_max;

    int percent = (adc_raw * 100) / adc_max;
    if(percent < 0) percent = 0;
    if(percent > 100) percent = 100;


    lv_bar_set_value(objects.bar1, percent, LV_ANIM_OFF);
    return percent;
}
```

Code listing 6 New file output_actions.h

```
/**
 * Convert a raw ADC reading into a bar percentage (0..100) and update the bar.
 *
 * - adc_raw: raw ADC counts (e.g. 0..4095 for 12-bit)
 * - adc_max: maximum ADC count (e.g. 4095 for 12-bit, 65535 for 16-bit)
 */
```

19. Now that we have defined the functions, we need to initialize the ADC and take the ADC reading that we will pass into the function that sets the bar percentage

Code listing 7 Modification to main.c to sample adc

```
//Add include
#include "output_actions.h"

// Additions to cm55_gfx_task function
static void cm55_gfx_task(void *arg)
{
    ...

    int32_t sar_adc_count;
    for (;;)
    {
        /* LVGL's timer handler function, to be called periodically to handle
         * LVGL tasks.
         */
        time_till_next = lv_timer_handler();
        vTaskDelay(pdMS_TO_TICKS(time_till_next));
        ui_tick();
        /* Read ADC result */
        sar_adc_count = Cy_AutAnalog_SAR_ReadResult(SAR_ADC_INDEX,
                        CY_AUTANALOG_SAR_INPUT_GPIO,SAR_ADC_CHANNEL);
        int percent=ui_set_bar_from_adc(sar_adc_count, 4095);
    }
}

//Additions in main to setup the SAR ADC add in after Enable global interrupts

    /* Initialize SAR ADC */
    result = Cy_AutAnalog_Init(&autonomous_analog_init);
    if (CY_AUTANALOG_SUCCESS != result)
    {
```

```
        handle_app_error();
    }

    /* Set interrupt mask to enable SAR0 result interrupt */
    Cy_AutAnalog_SetInterruptMask(CY_AUTANALOG_INT_SAR0_RESULT);

    /* Start autonomous control of the ADC */
    Cy_AutAnalog_StartAutonomousControl();

    /* Inform user about potentiometer connection */
    printf("ADC is configured in single channel configuration\r\n");
    printf("Provide the input voltage reading at pin P15_1,  \r\n");
    printf("which is connected to a potentiometer (POT)    \r\n\n");
```

20. Now users can build and program the application, when the potentiometer is turned on the EVK you will see the blue bar percentage change on the screen

4.6.3.1 Additional modifications

1. For this section of the lab we will provide high level explanations, and the user will have to take what they learned in the previous lessons to figure out the solution. The solution source code is provided with the lab materials
2. Now that we have the potentiometer controlling the bar on the gui it would be great to control the screen brightness using the potentiometer. To accomplish this find the function that sets the brightness of the display in the mtb_disp_dsi_waveshare_4p3 drivers file
3. The next task is to display text on the textArea that we added to the gui. Documentation can be found on the EEZ Studio website and LVGL Website. Note: The text area does have a max text length value that is set to 128 by default in EEZ Studio. Keep this in mind if the printed text is not seen
4. Solutions for this stretch goal can be found in the stretch goals folder under Lab 3

5 Graphics performance

5.1 Frames per second (FPS)

The frame per second (FPS) rendered by a graphics application is one of the key metrics to profile the performance of the application. In this section we go through the process of estimating the FPS.

5.1.1 Clock setup based on required FPS for video mode displays

The following set of formulas will be used to determine the Pixel and DSI clock required for given FPS. If the DSI clock is within the specification of display as well as MCU, the FPS can be supported.

- Total horizontal period

```
H_TOTAL = HSync width + HBP + Display active width + HFP [in pixels]
```

- Total vertical period

```
V_TOTAL = VSync width + VBP + Display active height + VFP [in lines]
```

- Pixel clock

```
PCLK = H_TOTAL × V_TOTAL × Frame Rate [in Hz]

Frame Rate == Frames per second (FPS in Hz)
```

- Bit clock (DSI clock)

```
Bit Clock = (PCLK × BPP) / (Number of Lanes)
```

5.1.2 FPS for command mode display

In case of command mode displays, the frame writes are limited by the MCU's DBI clock. The command mode displays contain a GRAM (Graphics RAM) which refreshes the display with its contents at the panels configured refresh rate. This setup allows asynchronous writes from the MCU to the display panel.

5.1.3 FPS improvement

Usage of GPU improves the FPS and reduces the CPU usage as compared to CPU rendered graphics.

The FPS can be further improved by efficient usage of GPU. Below are few techniques for the same:

- Use pre-multiplied images with alpha component. Alpha pre-multiply operation is not supported by GPU and is offloaded to CPU
- Prefer color formats which avoid conversion such as RGB565 (For UI) and ARGB8888 (For blend effects)

FPS can also be improved by proper memory placement. Following points summarize how to use memory effectively to improve FPS:

- Use double buffering with full resolution buffers
- Place the frame buffers in fast on-chip memory (SoCMEM) to reduce read/write latency for DC and GPU/CPU respectively
- Drive the CLK_HF2 (source clock for SoCMEM) at 300 MHz (max spec)
- Use the non-cacheable region of SoCMEM (gfx_mem) for frame buffers
- Align the stride and base address of each frame buffer to a 128-byte boundary to match AXI burst sizes and cache lines
- Place performance-critical routines (e.g., ISR handlers, blit/dispatch glue) in ITCM

Some of the techniques mentioned above are implemented in the Smartwatch code example for PSOC™ Edge E84. We recommend to go through this application to understand the implementation.

Also, the Smartwatch code example is thoroughly profiled in the application note:Available online**.**

5.2 LAB 3: Observe FPS/CPU usage with and without GPU

In this exercise we will observe the improvement in FPS of a graphics application when the graphics are CPU rendered versus when they are GPU rendered.

1. Create PSOC_Edge_Graphics_LVGL_Demo application
2. Set LV_USE_SYS_MON to 1 in *lv_conf.h*

Note: The application by default uses GPU rendering.

```
/*1: Enable system monitor component*/
#define LV_USE_SYSMON   1
```

3. Build and program
4. Observe that the FPS and CPU usage is logged in bottom left of the display. Note the readings
5. Now, to observe the performance for CPU rendered graphics, do the following changes:
   a. Set the LV_USE_DRAW_VG_LITE macro to 0 in *lv_conf.h*

```
/* Use VG-Lite GPU. */
#define LV_USE_DRAW_VG_LITE 0
```

   b. Open Device Configurator and disable the “Enable GPU” property as shown in Figure 26 and save the configuration

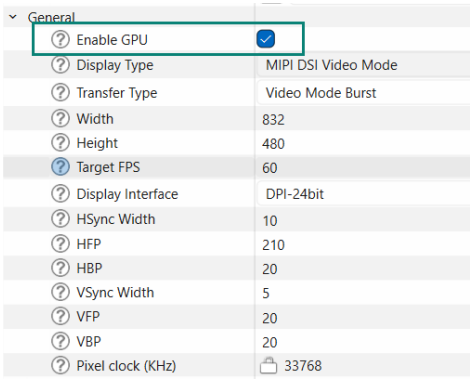


Figure 26 Disable GPU in device-configurator

   c. Comment out the following lines in the *main.c*

Code listing 8 Lines to comment in main.c

```
/*******************************************************************************
* Global Variables
*******************************************************************************/
/* Heap memory for VGLite to allocate memory for buffers, command, and
 * tessellation buffers
 */
```

```
// CY_SECTION(".cy_gpu_buf") uint8_t contiguous_mem[VGLITE_HEAP_SIZE] = { 0xFF };

// volatile void *vglite_heap_base = &contiguous_mem;
        /* Allocate memory for VGLite from the vglite_heap_base */
//        vg_module_parameters_t vg_params;
//        vg_params.register_mem_base = (uint32_t)GFXSS_GFXSS_GPU_GCNANO;
//        vg_params.gpu_mem_base[VG_PARAMS_POS] = GPU_MEM_BASE;
//        vg_params.contiguous_mem_base[VG_PARAMS_POS] = vglite_heap_base;
//        vg_params.contiguous_mem_size[VG_PARAMS_POS] = VGLITE_HEAP_SIZE;

        /* Initialize VGlite memory. */
//        vg_lite_init_mem(&vg_params);

        /* Initialize the memory and data structures needed for VGLite draw/blit
         * functions
         */
//        vglite_status = vg_lite_init((MY_DISP_HOR_RES) / 4,
//                                     (MY_DISP_VER_RES) / 4);
```

6. Build and run the application and note the FPS and CPU usage for GPU disabled scenario

5.2.1 Results

Table 5 Results with GPU enabled

–	Music paused	Music play
FPS (in Hz)	53-60	30-40
CPU usage (%)	47	~60

Table 6 Results with GPU disabled

–	Music paused	Music play
FPS (in Hz)	53-56	20-22
CPU usage (%)	50	77-92

6 Takeaways

- Understand the graphics subsystem’s configuration, in order to properly interface a display module with PSOC™ Edge E84 MCU
- Debug through a graphics application to understand the debug flow
- Understood RLAD decoder feature of display controller of the PSOC™ Edge E84 MCU
- Observed RL, RLA and RLAD decompression on PSOC™ Edge E84 MCU
- Understood FPS estimation and improvement techniques as well as identified opportunities for power reduction in graphics applications

7 Conclusion

This training provided an in-depth exploration of the graphics capabilities of the PSOC™ Edge platform from a system-level perspective. The course examined various features of the graphics subsystem in detail, equipping participants with the knowledge required to configure and optimize these features for specific application needs. Additionally, the training enhanced participants' ability to interpret and utilize related graphics collateral, promoting a deeper understanding and more effective use of the platform's graphics resources.

8 Appendix A: KIT_PSE84_EVAL details

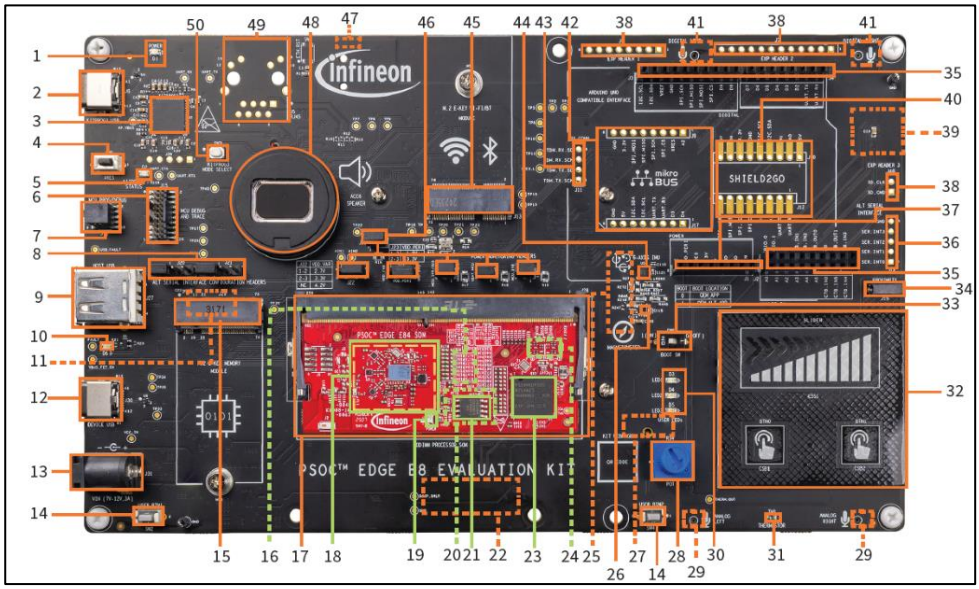


Figure 27 PSOC™ Edge EVK details

1 Base board power LED (D1)
2 KitProg3 program/debug USB Type-C connector (J8)
3 PSOC™ 5LP-based KitProg3 programmer and debugger (CY8C5868LTI-LP039, U2)
4 Reset button (SW1)
5 KitProg3 status LED (D2)
6 PSOC™ Edge E84 MCU ETM/JTAG debug and trace header (J15)
7 PSOC™ Edge E84 MCU 10-pin SWD/JTAG program and debug header (J16)
8 Alternative serial interface configuration headers (J20, J21)
9 PSOC™ Edge E84 MCU USB host Type-A connector (J27)
10 USB Type-C power delivery (PD) fault LED (D6)
11 Custom display capacitive touch panel connector (J37)**
12 PSOC™ Edge E84 MCU USB device Type-C connector (J30)
13 External power supply VIN connector (J31)
14 PSOC™ Edge E84 MCU user buttons (SW2, SW4)
15 M.2 (B-key) memory interface connector (J29)
16 128 Mbit Octal-SPI HYPERRAM™ (S70KS1283GABHI020, U12)***
17 Processor System on Module (SoM) 260-pin SODIMM connector (J28)
18 CYW55513 tri-band (Wi-Fi & Bluetooth®) combo radio (U3) section
19 Processor System on Module (SoM) power LED (D3)
20 1 Gbit Octal-SPI NOR flash (S28HS01GTGZBH1030, U10)***
21 128 Mbit Quad-SPI NOR flash (S25FS128SAGMFB100, U11)
22 MIPI-DSI custom display connector (J38)**
23 PSOC™ Edge E84 MCU (PSE846GPS2DBZC4A, U1)
24 PSOC™ 4000T CAPSENSE™ Co-processor (U9)***
25 Raspberry Pi compatible MIPI-DSI display connector (J39)**
26 3-axis magnetometer (U4)
27 Raspberry Pi compatible display capacitive touch connector (J41)**
28 Linear potentiometer (R34)
29 Analog microphones (IM73A135V01XTSA1, U36 and U37)**
30 User LEDs (D3, D4, D5)
31 Thermistor (TH1)
32 CAPSENSE™ buttons and slider (CSB1, CSB2, CSS1)
33 BOOT configuration switch (SW6)
34 Proximity sense connector (J19)
35 I/O headers compatible with Arduino UNO R3 (J2, J3, J4)
36 Alternative serial interface I/O header (J14)*
37 Power header compatible with Arduino UNO R3 (J1)
38 PSOC™ Edge E84 MCU expansion I/O headers (J6, J7, J40)*
39 MicroSD card holder (J35)**
40 Infineon's Shield2Go interface headers (J10, J12)*
41 PDM microphones (IM72D128V01XTMA1, U7 and U8)**
42 mikroBUS compatible headers by Mikroelektronika (J9, J17)*
43 Extended I2S header (J11)*
44 6-axis accelerometer and gyroscope IMU (U5)
45 M.2 (E-key) radio interface connector (J13)
46 PSOC™ Edge E84 MCU power selection/ monitoring headers (J18, J22, J23, J24, J25, J26)
47 Headphone connector (J34)*
48 Speaker (ACC6)
49 RJ45 ethernet magjack connector (J5)*
50 KitProg3 programming mode selection button (SW3)

*Footprint only, not populated on the board
**Component at the bottom side of the Baseboard
***Component at the bottom side of the SoM

Revision history

Document revision	Date	Description of changes
**	2026-04-22	Initial release.

Trademarks
All referenced product or service names and trademarks are the property of their respective owners.

Edition 2026-04-22
Published by

Infineon Technologies AG
Am Campeon 1-15
85579 Neubiberg
Germany

© 2026 Infineon Technologies AG. All Rights Reserved.

Do you have a question about this document?

Email:

erratum@infineon.com

Important Notice

Products which may also include samples and may be comprised of hardware or software or both ("Product(s)") are sold or provided and delivered by Infineon Technologies AG and its affiliates ("Infineon") subject to the terms and conditions of the frame supply contract or other written agreement(s) executed by a customer and Infineon or, in the absence of the foregoing, the applicable Sales Conditions of Infineon. General terms and conditions of a customer or deviations from applicable Sales Conditions of Infineon shall only be binding for Infineon if and to the extent Infineon has given its express written consent.

For the avoidance of doubt, Infineon disclaims all warranties of non-infringement of third-party rights and implied warranties such as warranties of fitness for a specific use/purpose or merchantability.

Infineon shall not be responsible for any information with respect to samples, the application or customer's specific use of any Product or for any examples or typical values given in this document.

The data contained in this document is exclusively intended for technically qualified and skilled customer representatives. It is the responsibility of the customer to evaluate the suitability of the Product for the intended application and the customer's specific use and to verify all relevant technical data contained in this document in the intended application and the customer's specific use. The customer is responsible for properly designing, programming, and testing the functionality and safety of the intended application, as well as complying with any legal requirements related to its use.

Unless otherwise explicitly approved by Infineon, Products may not be used in any application where a failure of the Products or any consequences of the use thereof can reasonably be expected to result in personal injury. However, the foregoing shall not prevent the customer from using any Product in such fields of use that Infineon has explicitly designed and sold it for, provided that the overall responsibility for the application lies with the customer.

Infineon expressly reserves the right to use its content for commercial text and data mining (TDM) according to applicable laws, e.g. Section 44b of the German Copyright Act (UrhG).If the Product includes security features:

Because no computing device can be absolutely secure, and despite security measures implemented in the Product, Infineon does not guarantee that the Product will be free from intrusion, data theft or loss, or other breaches ("Security Breaches"), and Infineon shall have no liability arising out of any Security Breaches.

If this document includes or references software:

The software is owned by Infineon under the intellectual property laws and treaties of the United States, Germany, and other countries worldwide. All rights reserved. Therefore, you may use the software only as provided in the software license agreement accompanying the software.

If no software license agreement applies, Infineon hereby grants you a personal, non-exclusive, non-transferable license (without the right to sublicense) under its intellectual property rights in the software (a) for software provided in source code form, to modify and reproduce the software solely for use with Infineon hardware products, only internally within your organization, and (b) to distribute the software in binary code form externally to end users, solely for use on Infineon hardware products. Any other use, reproduction, modification, translation, or compilation of the software is prohibited. For further information on the Product, technology, delivery terms and conditions, and prices, please contact your nearest Infineon office or visit https://www.infineon.com